Truthy and Falsy Values in JavaScript

Every value has a boolean side -- here's how to know which is which

In JavaScript, every single value -- whether it's a number, a string, an object, or anything else -- has an inherent boolean identity. When JavaScript needs a true/false answer (like inside an `if` statement), it doesn't complain if you hand it a string or a number. It just quietly converts it. Values that convert to `true` are called **truthy**, and values that convert to `false` are called **falsy**.

The good news: There are only 8 falsy values in all of JavaScript. Everything else is truthy. So if you memorize the short falsy list, you're covered.

The 8 Falsy Values -- memorize these

These are the only values in JavaScript that evaluate to false in a boolean context. That's it -- just eight of them. If a value isn't on this list, it's truthy.

Value	Type	What It Means	Example
`false`	Boolean	The literal boolean false	`if (false) { }`
`0`	Number	Zero -- the number, not the string	`if (0) { }`
`-0`	Number	Negative zero (yes, it exists)	`if (-0) { }`
`0n`	BigInt	BigInt zero	`if (0n) { }`
`""`	String	Empty string -- no characters at all	`if ("") { }`
`null`	Null	Intentional absence of any value	`if (null) { }`
`undefined`	Undefined	Declared but never assigned a value	`if (undefined) { }`
`NaN`	Number	Not a Number (failed math)	`if (NaN) { }`

Memory trick: Think of falsy values as "empty" or "nothing" values. No number (0), no text (""), no value at all (null, undefined), not even a valid number (NaN), and the literal false. If it feels like "nothing", it's probably falsy.

Truthy Values -- everything else

If a value isn't one of the 8 falsy ones listed above, it's truthy. Here are some common examples you'll run into regularly:

Value	Type	What It Means	Example
`true`	Boolean	The literal boolean true	`if (true) { }`
`1, 42, -5`	Number	Any non-zero number	`if (1) { }`
`"hello"`	String	Any non-empty string	`if ("hello") { }`
`[]`	Array	Empty array (it's an object)	`if ([]) { }`
`{}`	Object	Empty object	`if ({}) { }`
`function(){}`	Function	Any function reference	`if (fn) { }`
`new Date()`	Date	Date objects	`if (new Date()) { }`
`Infinity`	Number	Positive or negative infinity	`if (Infinity) { }`

The Ones That Catch People Off Guard

These values look like they should be falsy, but they're not. This is where a lot of bugs come from, so pay attention to these:

Value	Why It's Truthy	The Mistake People Make
`"0"`	It's a non-empty string (has one character)	Thinking string "0" is falsy like number 0
`"false"`	It's a non-empty string (has 5 characters)	Thinking string "false" behaves like boolean false
`[]`	Arrays are objects, and objects are always truthy	Expecting an empty array to be falsy
`{}`	Objects are always truthy, even empty ones	Expecting an empty object to be falsy
`" "`	Contains a space character -- it's not empty	Thinking whitespace-only strings are falsy

The key insight: For strings, "empty" means zero characters -- not "looks empty" or "looks false". The string `"0"` has one character in it, so it's truthy. The string `"false"` has five characters, so it's truthy too. For objects and arrays, they're always truthy regardless of what's inside them.

Seeing It in Action

```
// Checking if a string has content
let username = "";
if (username) {
    console.log("Welcome!");
} else {
    console.log("Please enter a name");  // This runs
}

// Watch out for "0" though!
let input = "0";
if (input) {
    console.log("Has value");   // This runs -- "0" is truthy!
}

// Checking if an array has items (don't rely on truthiness)
let items = [];
if (items) {
    console.log("Truthy!");     // This runs -- [] is truthy!
}
if (items.length) {
    // This is the right way to check for empty arrays
}
```

Pro tip: Don't use `if (array)` to check if an array has items -- it will always be true. Use `if (array.length)` instead. Similarly, for objects, use `if (Object.keys(obj).length)` to check if an object has properties.

Remember: only 8 values are falsy, everything else is truthy. When in doubt, test it with Boolean(value) in the console to see what JavaScript thinks.